

EPAM Technical Interview Preparation Guide

Node.js / JavaScript (Fullstack)

Senior Software Engineer

Generated: 2026-07-10

Prepared with Codemie AI for EPAM interview preparation

EPAM internal level: A3 (requested as L3)

Purpose of This Guide

This document helps you prepare for an EPAM technical interview for a Senior Node.js/JavaScript Fullstack profile. It highlights the areas most commonly evaluated at senior level: production-ready coding, system and module design, quality practices, troubleshooting, and pragmatic architecture trade-offs.

How to Use This Guide

1. Skim the 'What to Expect' section to understand interview structure and depth.
2. Use 'Technical Interview Focus Areas' to prioritize your study time.
3. For each topic in 'Detailed Topics to Review', prepare: (1) definitions, (2) examples from your projects, (3) typical pitfalls.
4. Practice the 'Sample Technical Questions' out loud; aim for structured answers (context → decision → trade-offs → result).
5. In the final 48 hours, follow the 'Final Preparation Checklist'.

Role and Level Overview

Senior engineers at EPAM are expected to work with minimal supervision, take ownership of implementation and design decisions, mentor others, and explain trade-offs (performance, security, reliability) clearly. You should be comfortable designing and operating Node.js services in production contexts and collaborating across teams.

What to Expect in the Interview

- Format is often a mix of: core Node.js/JavaScript questions, practical coding (API/logic), debugging/troubleshooting, and design discussion.
- Expect senior-level depth: not only 'how', but 'why'—trade-offs, failure modes, observability, and maintainability.
- You may be asked about asynchronous behavior, event loop implications, performance bottlenecks, security risks, and deployment practices.
- If you don't know an answer: state assumptions, propose how you would validate (logs, metrics, docs), and outline a safe approach.
- Interviewer Focus: clarity of reasoning, production thinking, and ability to guide others (not just personal coding speed).

Technical Interview Focus Areas

1) Node.js runtime mastery

- Event loop & non-blocking model; how async I/O differs from CPU-bound workloads.
- Streams, backpressure, buffers; when to avoid loading full payloads into memory.
- Process lifecycle: environment configuration, exit codes, stdout/stderr, signals.
- Modules: CommonJS vs ES Modules; package entry points; dependency management.

2) Backend service development (frameworks & APIs)

- Routing/middleware lifecycle; structured error handling; input validation.
- Authentication/authorization patterns (JWT, Passport strategies) and secure session/token handling.
- API design: REST conventions, pagination/filtering, idempotency, versioning.
- Real-time patterns: WebSockets/Socket.IO and operational considerations.

3) Data access & consistency

- Relational vs NoSQL trade-offs; connection pooling; transaction usage.
- ORM/ODM or query builders; parameterized queries to mitigate injection.
- Indexing awareness and performance implications; handling retries and transient failures.

4) Testing strategy & quality

- Unit/component/integration tests; when to add contract tests.
- TDD/BDD basics in practice; test isolation and determinism.
- E2E + performance/stress testing fundamentals; what 'good coverage' means beyond %.
- Interviewer Focus: ability to explain how you guarantee quality and manage technical debt.

5) Performance, debugging, and troubleshooting

- Profiling and diagnosing event-loop stalls; async debugging; memory leak patterns.
- Choosing sync vs async APIs; CPU-heavy work offloading (worker threads / clustering where appropriate).
- Production troubleshooting stories: how you narrowed down root cause using logs/metrics/traces.

6) Security fundamentals (expected at senior level)

- OWASP awareness and common attack vectors (XSS, CSRF, SSRF, injection, traversal).

- Secure transport (HTTPS/TLS); secrets handling; dependency risk (npm audit).
- Defense-in-depth: validation, encoding, least privilege, secure headers (e.g., Helmet).

7) Deployment & cloud readiness

- CI/CD basics, npm scripts as build tooling; environment-based configuration.
- Containerization practices (Docker, Compose); image building and runtime configuration.
- Cloud integration concepts: load balancing, autoscaling, managed DB/cache, health checks.
- Observability: logs/metrics/alerts; dashboards (Grafana/Kibana/Prometheus/Zabbix).

8) Architecture and patterns

- Application structure principles; modularization; layering and boundaries.
- GoF patterns (creational/structural/behavioral) and how they show up in Node.js services.
- Microservices basics and more advanced practices: consistency patterns and trade-offs.
- Availability vs consistency; performance vs scalability—be ready to articulate choices.

Detailed Topics to Review

Node.js Core (Advanced)

- Event loop phases; timers vs I/O callbacks; promise microtasks; practical implications.
- Streams (readable/writable/transform) and backpressure; piping large uploads/downloads.
- Buffers and memory considerations; avoiding unbounded buffering.
- Child processes & IPC basics; when to use them.
- Diagnostics: module loading issues, runtime errors, async stack traces, heap snapshots.

Frameworks (Advanced usage)

- Middleware ordering and error middleware; consistent error responses.
- Validation and schema (e.g., JSON schema) and handling partial failures.
- Extensibility hooks/lifecycle; designing maintainable modules/services.
- Security and logging within the framework (request IDs, structured logs).

Authorization and Security

- JWT: signing vs encryption, expiry/refresh, rotation; typical mistakes.
- OAuth at a high level (flows) if applicable; third-party auth integration.

- SSRF risk in server-side HTTP calls; allowlists and egress controls.
- Dependency scanning, npm audit workflow; mitigation strategy.

Queues & async processing

- When to use a queue; at-least-once vs exactly-once thinking.
- Basic awareness of RabbitMQ/ActiveMQ/Kafka/SQS/SNS; retries and dead-lettering.
- Failover techniques and idempotent consumers.

Logging, monitoring, visualization

- Log levels and structured logging; correlation IDs.
- Tools: Winston/Logstash; remote logging; dashboarding via Kibana/Grafana.
- Defining useful alerts: symptoms (latency, error rate, saturation) vs causes.

Engineering Excellence

- Code quality and metrics: what you measure and why (complexity, duplication, coverage trends).
- Code review approach: correctness, readability, security, operability.
- Technical debt management: prioritization and communication to stakeholders.

What the Interviewer May Look For

- Ownership: you can drive a feature from design to production readiness.
- Trade-offs: you compare options (e.g., queue vs direct calls; monolith vs microservice) with clear criteria.
- Production thinking: logging/metrics, failure handling, config/secrets, deployment approach.
- Quality mindset: layered testing strategy, review discipline, preventing regressions.
- Mentoring: how you explain concepts and help less experienced engineers.

Practical Preparation Tips

- Pro Tip: Prepare 2–3 short ‘success stories’ (debugging, performance, incident, refactor) and practice telling them with metrics and outcomes.
- Build a small Node.js API and add: validation, JWT auth, structured logging, tests, and a Dockerfile—then deploy it (even to a sandbox).

- Practice explaining the event loop with a concrete example (e.g., `setTimeout` + promises + I/O).
- Rehearse how you'd investigate: memory leak, p95 latency regression, sudden spike in 500s.

Common Mistakes to Avoid

- Common Pitfall: treating Node.js as 'single-threaded therefore slow'—instead explain I/O vs CPU and mitigation (workers/clustering).
- Common Pitfall: shallow security answers (e.g., 'we use JWT') without discussing token lifecycle, storage, and validation.
- Common Pitfall: no error strategy—unclear boundaries for retries, timeouts, and user-facing errors.
- Common Pitfall: tests that mock everything (low value) or no integration tests for critical flows.
- Common Pitfall: logging too much or too little; no correlation IDs; sensitive data in logs.

Sample Technical Questions

Node.js Core & Async

6. Explain the event loop. How do promise microtasks interact with timers and I/O callbacks?
7. How do streams and backpressure work? Give an example where streams prevent an outage.
8. How would you debug a memory leak in a Node.js service? What tools and signals would you use?

API Design & Security

9. Design an endpoint for listing resources with pagination, filtering, and sorting. What are the trade-offs?
10. JWT-based auth: how do you handle refresh, revocation, and clock skew?
11. What is SSRF and how can it occur in a Node.js service? How would you mitigate it?

Data & Consistency

12. When do you choose SQL vs NoSQL for a new service?
13. How do transactions work in your chosen DB, and how do you map them in Node.js code?
14. How would you design idempotent processing for a queue consumer?

Testing & Delivery

15. Describe your testing pyramid for a Node.js service. Where do contract tests fit?
16. How do you ensure code quality in CI/CD? Which checks are 'must-have' and why?
17. What's your strategy for handling technical debt without stopping feature delivery?

Final Preparation Checklist

18. Review: event loop, streams/backpressure, common debugging approaches.
19. Review: API error handling, validation, JWT/OAuth basics, OWASP top risks.
20. Review: DB fundamentals (pooling, transactions, indexing) and one ORM/ODM you know.
21. Practice: 2 coding exercises (API logic + async) and 1 debugging scenario.
22. Prepare: 2–3 project stories with clear impact (latency, cost, reliability, incidents).
23. Ensure: your environment is ready (IDE, node version familiarity, lint/test commands).

Closing Advice

Aim to demonstrate senior-level thinking: clear assumptions, structured problem solving, trade-off awareness, and production readiness. During the interview, narrate your reasoning, validate edge cases, and connect answers to real project experience.